CONFIDENTIAL. Limited circulation. For review only.

Why Smooth Randomly if You Can Smooth Principledly: Locally Smoothed Trajectory Optimization for Non-Smooth Robotic Tasks

author names omitted for anonymous review

***Abstract*— Contact-rich tasks in robotics, such as manipulation and locomotion, fundamentally challenge trajectory optimization due to non-smooth dynamics and frequent mode transitions. Traditional derivative-based methods like DDP, while being sample efficient, struggle with non-smooth dynamical behavior. In contrast, standard reinforcement learning approaches, whose stochastic nature has been suggested to enable more effective exploration of the solution space by averaging and thus reasoning over multiple scenarios at once, are surprisingly successful. These approaches although come at the cost of sample efficiency. Recently, new methods have emerged that attempt to bridge these complementary strengths–sample efficiency and stochasticity. A common thread among them is the incorporation of *randomized smoothing* and the careful design of *stochasticity scheduling*, though each approach addresses these components in a different way. This work reviews and relates these aspects, with particular attention to a method that provides a *principled* formulation of both, which we also apply to non-smooth robotic tasks and evaluate alongside other approaches. We highlight the different strengths these approaches offer and indicate when each is most effective. Together, these methods demonstrate automatic mode discovery and feasible motion synthesis for systems with non-smooth dynamics, advancing contact-implicit planning for robotic tasks. Our implementation is available at: `https://github.com/marginallyunstable/topoc`**

I. Introduction

As robots transition from controlled laboratory settings to real-world environments, they are increasingly required to interact with their surroundings in purposeful and sustained ways. Tasks such as object manipulation, tool use, and locomotion over uneven terrain demand reasoning about intermittent contact, frictional forces, and complex dynamics. Addressing these challenges has brought contact-rich planning and control to the forefront of modern robotics research. An important aspect of addressing these control challenges is the problem of trajectory optimization (TO), which focuses on generating dynamically feasible and efficient motions. Taking advantage of the first/second order approximations of the dynamics and cost function, algorithms like Differential Dynamic Programming (DDP) [1] are able to come up with optimal trajectories for complex systems very efficiently. While many algorithms have been proposed to solve TO problems, DDP has remained popular due to its straightforward implementation, computational efficiency, dynamic feasibility for every iteration, and the ability to produce a time-varying linear feedback policy as a byproduct of its optimization [2]. These reasons along with the algorithm's numerous variations and extensions, make it especially appealing as an online controller [3], [4]. However, non-smooth phenomena are ubiquitous in robotics. Contact-rich manipulation and locomotion, for example, are replete with impacts, making-and-breaking of contact, and dry friction. Unsurprisingly, these algorithms fail when dealing with such non-smooth dynamical systems, as they induce non-informative or discontinuous gradients [5].

Derivative-free or zero-th order approaches like standard Reinforcement Learning (RL), on the other hand have shown tremendous progress in robotics, particularly for tasks involving contact-rich planning and control [6], [7]. Despite demonstrating strong performance, standard RL is often sample hungry, illustrating the no-free-lunch trade-off: disregarding derivative information comes at the cost of reduced sample efficiency.

Recent research investigating RL's effectiveness in non-smooth settings attributes its success to the intrinsic stochastic nature of its learning process, which has been observed to perform a mechanism akin to *randomized smoothing* [8], [9]. Broadly speaking, this stochasticity averages over and smooths out the non-smoothness inherent in the problem and also allows to escape non-informative gradients, enabling effective derivative-based updates. These insights have inspired efforts to combine the strengths of RL and derivative-based optimal control, to tackle TO for non-smooth dynamical systems [10], [11].

Concurrent to the aforementioned works, improvements on the DDP algorithm have continued. In particular, the Fourier-Hermite Dynamic Programming (FHDP), a zero-th order approach, extends classical DDP by representing the value and state-action value functions using Fourier-Hermite series expansions instead of the classical Taylor series expansion of the cost and dynamics function [12]. Avoiding direct derivative calculations, FHDP provides a *principled* way to use stochastic approximation for indirect calculation of (smooth) derivatives within the DDP framework. As one might expect, a key consideration when incorporating this stochasticity, is how it is scheduled over the algorithm iterations. It must be sufficient initially to smoothen the problem and encourage exploration, while decaying gradually to allow exploiting the learned structure of the problem and converge to an optimal trajectory. A similar observation applies to the works on smoothed TO mentioned earlier.

Looking at deterministic trajectory optimization (TO) problems through the lens of the emerging paradigm of Control as Inference [13]–[16], previous work has demonstrated how to derive an algorithm similar to DDP through a probabilistic reformulation of a deterministic TO problem [17], [18]. Leveraging its probabilistic formulation, the algorithm inherently uses stochasticity as a means of exploration.

Manuscript 1204 submitted to 2026 IEEE International Conference on Robotics and Automation (ICRA). Received September 16, 2025.

In particular, [18] further incorporated ideas from FHDP, which naturally achieve the desired smoothing properties while still ensuring deterministic execution. Moreover, their method addressed the aforementioned issue of *stochasticity scheduling* in a *principled* manner, enabling both efficient exploration of the optimization landscape and accelerated convergence across a range of nonlinear systems.

So, why smooth randomly if you can smooth principledly? Our paper explores this question in the context of trajectory optimization for non-smooth robotic tasks. Our contributions are the following:

- We examine how three representative methods [10], [11], [18] implement *randomized smoothing* and *stochasticity scheduling*, analyzing their similarities, differences, strengths, and practical trade-offs.
- We extend the principled framework of [18] to non-smooth robotic tasks, and augment it with three practical schemes for scheduling stochasticity.
- We benchmark these approaches on simple yet representative non-smooth robotic tasks, showing how they enable automatic discovery of mode sequences and the synthesis of feasible motions.

The remainder of the paper is organized as follows. Sec. II reviews trajectory optimization with DDP, the challenges posed by non-smooth dynamics, and recent insights to address them. Sec. III discusses extensions of DDP from [10] and [11] for non-smooth robotic tasks. Sec. IV introduces the probabilistic optimal control framework and summarizes key results from [18]. Sec. V presents simulations, and Secs. VI–VII conclude with discussion and future directions.

II. Background

In this section, we present a brief background on TO via the widely used DDP algorithm. We then discuss the challenges posed by non-smooth problems in robotics and highlight randomized smoothing, a recent technique increasingly adopted in robotics to tackle these challenges.

A. Trajectory Optimization via DDP/iLQR

Consider the deterministic trajectory optimization problem

$$\begin{aligned} \min_{u_{0:T-1}} \quad & C(\xi_{0:T}) \\ \text{s.t.} \quad & x_{t+1} = f_t(\xi_t), \quad t = 0, \dots, T-1. \end{aligned} \tag{1}$$

where, $\xi_t = (x_t, u_t)$, $C(\xi_{0:T}) := c_T(x_T) + \sum_{t=0}^{T-1} c_t(\xi_t)$ is the cumulative cost and $f_t(\xi_t)$, $c_t(\xi_t)$ and $c_T(x_T)$ are dynamics function, the stage and terminal costs, respectively.

DDP proceeds by iteratively approximating the state-action value function $Q_t(\xi_t)$ and the value function $V_t(x_t)$ quadratically around a nominal trajectory $\hat{\xi}_{0:T}$ [1]:

$$Q_t(\xi_t) \approx \hat{Q}_t(\xi_t) = \frac{1}{2} \begin{bmatrix} 1 \\ \delta\xi_t \end{bmatrix}^\top \begin{bmatrix} 2\hat{Q}_{0,t} & \hat{Q}_{\xi,t}^\top \\ \hat{Q}_{\xi,t} & \hat{Q}_{\xi\xi,t} \end{bmatrix} \begin{bmatrix} 1 \\ \delta\xi_t \end{bmatrix}, \tag{2}$$

$$V_t(x_t) \approx \hat{V}_t(x_t) = \frac{1}{2} \begin{bmatrix} 1 \\ \delta x_t \end{bmatrix}^\top \begin{bmatrix} 2\hat{V}_{0,t} & \hat{V}_{x,t}^\top \\ \hat{V}_{x,t} & \hat{V}_{xx,t} \end{bmatrix} \begin{bmatrix} 1 \\ \delta x_t \end{bmatrix}, \tag{3}$$

where $\delta\xi_t = \xi_t - \hat{\xi}_t$, and $\delta x_t = x_t - \hat{x}_t$.

[1]For notational convenience we write $\xi_{0:T}$ when we mean $\{\xi_{0:T-1}, x_T\}$.

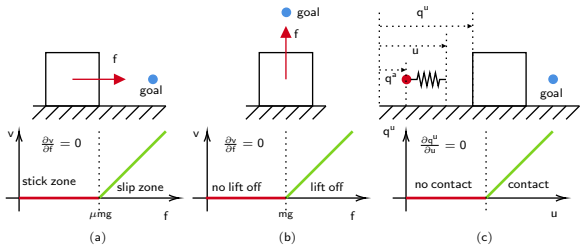

Fig. 1. Schematic representations and corresponding state-input diagrams of simple non-smooth dynamical systems. (a) Block of ground with dry Coulomb friction (b) Block sitting ground making a unilateral contact (c) A sphere pushing a block (abstracting a robot end-effector pushing a object). Notation: f-force, v-velocity, mg-weight of block, μ-friction coefficient, u displacement input, (q^u, q^a)-block and sphere position.

The coefficients of $\hat{Q}_t$ are computed recursively in a backward pass via:

$$\hat{Q}_{0,t} = \hat{c}_{0,t} + \hat{V}_{0,t+1}, \tag{4a}$$

$$\hat{Q}_{\xi,t} = \hat{C}_{\xi,t} + \hat{F}_{\xi,t}^\top \hat{V}_{x,t+1}, \tag{4b}$$

$$\hat{Q}_{\xi\xi,t} = \hat{C}_{\xi\xi,t} + \hat{F}_{\xi,t}^\top \hat{V}_{xx,t+1} \hat{F}_{\xi,t} + \hat{V}_{x,t+1} \otimes \hat{F}_{\xi\xi,t}, \tag{4c}$$

where $\hat{c}_{0,t}, \hat{C}_{\xi,t}, \hat{C}_{\xi\xi,t}$ are the value, gradient, and Hessian of c_t at $\hat{\xi}_t$. $\hat{F}_{\xi,t}, \hat{F}^i_{\xi\xi,t}$ are the Jacobian and Hessian of f_t and its components and $\otimes$ denotes a tensor product.

Minimizing (2) over δu_t provides the incremental control:

$$\delta u_t^* = \underbrace{-\hat{Q}_{uu,t}^{-1} \hat{Q}_{u,t}}_{k_t^*} + \underbrace{-\hat{Q}_{uu,t}^{-1} \hat{Q}_{ux,t}}_{K_t^*} \delta x_t. \tag{5}$$

where k_t^* and K_t^* can be seen as the feedforward and feedback gains respectively.

When this control is substituted in (2), the following quadratic approximation of the value function can be constructed:

$$\hat{V}_{0,t} = \hat{Q}_{0,t} - \tfrac{1}{2} k_t^{*\top} \hat{Q}_{uu,t} k_t^* \tag{6}$$

$$\hat{V}_{x,t} = \hat{Q}_{x,t} - K_t^{*\top} \hat{Q}_{uu,t} k_t^* \tag{7}$$

$$\hat{V}_{xx,t} = \hat{Q}_{xx,t} - K_t^{*\top} \hat{Q}_{uu,t} K_t^* \tag{8}$$

A forward pass simulates the system using the updated policy, producing a new nominal trajectory and this process repeats until convergence.

Omitting the last term in (4c) (i.e. the second-order information from the dynamics), gives us the well-known iLQR algorithm. We refer the reader to [19] for further details on the DDP algorithm.

B. Challenges with non-smooth trajectory optimization

To illustrate the challenges of derivative-based trajectory optimization techniques for contact-rich tasks, we consider three toy examples borrowed from [8] and [20] (Fig. 1). In all three cases, derivative information of the dynamics is null during: sticking, no lift off and no contact mode in (a), (b) and (c) respectively, leading to null gradient Q_u, and thus trapping derivative-based algorithms at a local-minima [11]. Moreover, transitions between modes result in discontinuous derivatives, making derivative-based optimization unreliable.

Manuscript 1204 submitted to 2026 IEEE International Conference on Robotics and Automation (ICRA). Received September 16, 2025.

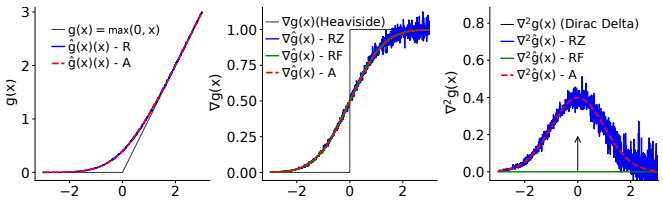

Fig. 2. Derivative estimates (Middle: gradient, Right: Hessian) of the ReLU function using randomized smoothing (R: Randomized, A: Analytical, RZ: Randomized Zero-order, RF: Randomized First-order)

C. Randomized Smoothing

To address the susceptibility of gradient descent for non-smooth function optimization, gradient sampling methods were developed, which stabilize the optimization process by constructing a descent direction from the convex hull of sampled gradients [21], [22]. Optimal convergence rates and guarantees were studied and a zero-order variant was proposed in [23], where the approach was also referred to as *randomized smoothing* (RS). For a detailed presentation of RS, including formal definitions and theoretical properties, we refer the reader to [10]. To provide essential context, we briefly review the first and zero-th order variants of derivative estimators using RS.

Let $g : \mathbb{R}^n \to \mathbb{R}^m$ be a locally Lipschitz (possibly non-smooth) function. Consider a random vector $\epsilon \sim \mathcal{N}(0, I)$ in $\mathbb{R}^n$, and let Σ denote a covariance matrix. A smoothed version of g can be given as follows:

$$\hat{g}(x) := \mathbb{E}[g(x + C\epsilon)] \tag{9}$$

where, C is the Cholesky factor of a covariance matrix Σ.

Using integration by parts, we arrive to the following expressions of the gradient of $\hat{g}$:

$$\nabla_x \hat{g}(x) = \mathbb{E}\left[\nabla_x g(x + C\epsilon)\right] \tag{10a}$$
$$= \mathbb{E}\left[g(x + C\epsilon) \otimes C^{-1}\epsilon\right] \tag{10b}$$

where (10a) and (10b) correspond respectively to the first and zero-th order variant for the smoothed gradient of g and can be estimated with a Monte-Carlo estimator.

The Hessian of $\hat{g}$ can be estimated as follows using randomized smoothing [24]:

$$\nabla_x^2 \hat{g}(x) = \mathbb{E}_{\epsilon\sim\rho}\left[\nabla_x^2 g(x + C\epsilon)\right] \tag{11a}$$
$$= \mathbb{E}_{\epsilon\sim\rho}\left[\nabla_x g(x + C\epsilon) \otimes C^{-1}\epsilon\right] \tag{11b}$$
$$= \mathbb{E}_{\epsilon\sim\rho}\left[g(x + C\epsilon) \otimes C^{-1}(\epsilon\epsilon^\top - I)C^{-1}\right] \tag{11c}$$

Fig.2 shows an example of gradient and Hessian estimation using zero-th and first order RS of the ReLU function (similar to the dynamics functions we see in examples of Fig.1). As can be seen in the gradient and Hessian plots, the smoothed derivatives are informative even in the flat region of $g(x)$ (i.e. $x < 0$). One can also note that the zero-th order estimate exhibits higher variance as compared to the first-order estimate. Although, the first order estimate comes with its own drawback. As was pointed out in previous work, and is clear from the Hessians plot, the Monte-Carlo estimate using first order gradient fails to converge to the gradient of $\nabla\hat{g}(x)$ (sampling from the Dirac-delta is ill-defined, as it does not evaluate pointwise and in practice leads to 0 almost surely) [8, Theorem 1].

While zero-order RS estimation may exhibit higher variance, it has the advantage of being applicable even when derivatives are unavailable or difficult to compute, which is the case for the dynamics function in contact-rich problems. Unlike many differentiable simulators for modeling contact that rely on hand-crafted smoothing of dynamics [25], randomized smoothing in the zero-order form provides an implicit way to handle non-smooth dynamics without requiring explicit approximations.

III. DDP/iLQR with Randomized Smoothing

As mentioned before, earlier works proposed TO algorithms for contact-rich tasks in robotics, by incorporating smoothed versions of the dynamics function and its derivatives using randomized smoothing [10] and [11]. This was done within the framework of the DDP algorithm, and referred to as the Randomized DDP algorithm in [11], while a variant of the iLQR algorithm, which was termed iMPC[2], was proposed in [10].

While both works share the same underlying philosophy, there is a key distinction based on the dimension where smoothing is performed. The first approach exists in applying smoothing in both the state and control spaces [10], while the second approach restricts smoothing to the control dimension [11]. The rationale for this restriction, as argued by [11], is that perturbing only the control ensures that only reachable states (e.g. ones that meet non-penetration constraints for contact) are explored. They further established connections between reinforcement learning and optimal control through the lens of randomized smoothing, motivating a practical choice: interpreting randomized smoothing as an exploration term and smoothing only the control space. This, they argue, reduces the sampled space dimension while still achieving strong performance on complex robotic tasks.

We agree with [11] that perturbing only the control effectively explores reachable states. However, as noted in [26], computing the mean of these perturbed samples during the forward pass can still lead to infeasible states. Additionally, using a bundled estimate in the state space, i.e. aggregating information from many perturbed states rather than relying on a single point, can provide smoother and more stable estimates of the dynamics and its derivatives. It is also worth noting that many recent robotics simulators solve contact dynamics implicitly as an optimization problem wherein non-penetration constraints are imposed [20], [27], [28]. As discussed in [10], reasoning about dynamics at an infeasible (penetrating) configuration may seem ill-posed; however, the dynamics can be defined by projecting the infeasible configuration onto the nearest feasible (non-penetrating) point. Taken together, these observations suggest that both strategies, *control-only smoothing* and *state-and-control smoothing*, involve their own trade-offs. In what follows, we describe these two variants, referring to them as *State-Control-Smoothed DDP (SCS-DDP)* and *Control-Smoothed DDP (CS-DDP)*.

[2]This was done to incorporate state and input constraints in the TO problem. In principle, if the constraints are omitted, one would get the standard iLQR algorithm.

A. State-Control-Smoothed DDP (SCS-DDP)

The State-Control-Smoothed DDP variant can be obtained simply by substituting the dynamics function and its derivatives by their respective smoothed versions within the backward and forward pass of the DDP algorithm. Accordingly, we focus on the equations for the smoothed dynamics and its derivatives. Borrowing the same notation from Sec. II-A and II-C, the smoothed dynamics function and its derivatives can be given as follows (Only the zeroth-order variants are described, higher-order variants can be obtained using equations (10) and (11):

$$\hat{f}_t(\xi_t) := \mathbb{E}\big[\, f_t(\xi_t + C_{\xi\xi,t}\,\epsilon)\,\big], \tag{12a}$$

$$\hat{F}_{\xi,t}(\xi_t) = \mathbb{E}\big[\, f_t(\xi_t + C_{\xi\xi,t}\epsilon)\ \otimes\ C^{-1}_{\xi\xi,t}\epsilon\,\big], \tag{12b}$$

$$\hat{F}_{\xi\xi,t}(\xi_t) = \mathbb{E}\Big[\, f_t(\xi_t + C_{\xi\xi,t}\epsilon)\ \otimes\ C^{-1}_{\xi\xi,t}\big(\epsilon\epsilon^\top - I\big)C^{-1}_{\xi\xi,t}\,\Big] \tag{12c}$$

where, the random vector $\epsilon \sim \mathcal{N}(0, I)$ is in $\mathbb{R}^{n_x+n_u}$ and $C_{\xi\xi,t}$ denotes the Cholesky factor of the covariance matrix $\Sigma_{\xi\xi,t}$ associated with the combined vector $\xi_t = (x_t, u_t)$.

B. Control-Smoothed DDP (CS-DDP)

For Control-Smoothed DDP, as is done in Section III-A, we provide the smoothed dynamics and their derivatives under control-only smoothing:

$$\hat{f}_t(x_t, u_t) := \mathbb{E}\big[f_t(x_t, \tilde{u}_t)\big], \tag{13a}$$

$$\nabla_x \hat{f}_t(x_t, u_t) = \mathbb{E}\big[\nabla_x f_t(x_t, \tilde{u}_t)\big], \tag{13b}$$

$$\nabla^2_x \hat{f}_t(x_t, u_t) = \mathbb{E}\big[\nabla^2_x f_t(x_t, \tilde{u}_t)\big], \tag{13c}$$

$$\nabla_u \hat{f}_t(x_t, u_t) = \mathbb{E}\big[f_t(x_t, \tilde{u}_t) \otimes C^{-1}_{uu,t}\epsilon\big], \tag{13d}$$

$$\nabla_{ux} \hat{f}_t(x_t, u_t) = \mathbb{E}\big[\nabla_x f_t(x_t, \tilde{u}_t) \otimes C^{-1}_{uu,t}\epsilon\big], \tag{13e}$$

$$\nabla^2_u \hat{f}_t(x_t, u_t) = \mathbb{E}\Big[f_t(x_t, \tilde{u}_t) \otimes C^{-1}_{uu,t}(\epsilon\epsilon^\top - I)C^{-1}_{uu,t}\Big]. \tag{13f}$$

where, $\tilde{u} := u_t + C_{uu,t}$, the random vector $\epsilon \sim \mathcal{N}(0, I)$ is in $\mathbb{R}^{n_u}$ and $C_{uu,t}$ denotes the Cholesky factor of the covariance matrix $\Sigma_{uu,t}$ associated with the input u_t.

Note that computing the smoothed gradient (13b) and Hessians (13c), (13e) of f_t using this approach requires access to the derivatives of f_t, which may not always be available.

C. Covariance Scheduling

As mentioned earlier in Sec. I, a key aspect of introducing stochasticity in the TO problem is to properly schedule it: enough to smooth the non-smooth optimization landscape and overcome non-informative derivatives, while gradually decaying it to eventually solve the original non-smooth problem.

Following Robbins-Monro rule [29], the authors in [10] decrease the covariance $\Sigma_{\xi\xi,t}$ every iteration of their algorithm, whereas [11] proposed solving a cascade of smoothed DDP problems, reducing the covariance $\Sigma_{uu,t}$ for each successive problem. To maintain a consistent framework for comparisons later, we adopt the cascade-based covariance scheduling approach of [11] for both SCS-DDP and CS-DDP, and summarize the SCS-DDP algorithm in Alg. 1. The CS-DDP is then analogous to SCS-DDP, with $\Sigma_{\xi\xi,t}$ replaced by $\Sigma_{uu,t}$[3]. The parameters α, β, and γ denote the precision threshold for terminating each smoothed DDP problem in the cascade, the covariance decay factor, and the precision decay factor, respectively. The quantity ΔC denotes the change in the cumulative cost, computed as the difference between its value in the current and previous iteration.

Algorithm 1: SCS-DDP algorithm

Input: TO problem: $C(\xi_{0:T}), f_t(\xi_t)$, initial trajectory: $\xi_{0:T}$, target control covariance and precision: $\Sigma^*_{\xi\xi}, \alpha^*$, initial control covariance and precision: $\Sigma_{\xi\xi,0}, \alpha_0$, adaptive scheme parameters: β, γ;
Output: $u^*_{0:T-1}$;
repeat
  repeat
    $k^*_{0:T}, K^*_{0:T}$: ▷ Backward Pass using (5);
    $\xi_{0:T}$: ▷ Forward Pass;
  until $|\Delta C| < \alpha$;
  $\Sigma_{\xi\xi} \leftarrow \Sigma_{\xi\xi}/\beta$;
  $\alpha \leftarrow \alpha/\gamma$;
until $\alpha < \alpha^*$ *and* $\Sigma_\xi \prec \Sigma^*_\xi$;

IV. Probabilistic Differential Dynamic Programming

In this section, we summarize the approach taken in the work [18], for efficiently solving deterministic TO problems using the probabilistic optimal control (POC) paradigm [13], [14]. In [18], the motivation for introducing stochasticity into the TO problem came from a different route than that of [10] and [11]. POC casts the Risk-Sensitive Stochastic Optimal Control (RSOC) problem as a probabilistic inference problem, making it amenable to well-established inference techniques, particularly the Expectation-Maximization (EM) algorithm. Building on the insight that for deterministic dynamics, the solution of the TO problem (1) and RSOC coincide [18, Lemma 1], an efficient EM-based algorithm, structurally similar to DDP, can be derived, which we refer to as Probabilistic-DDP throughout this paper. The formal definition of RSOC and a detailed description of the algorithm are provided in the subsequent subsections. We would like to point out that a key distinction of the approach in [18] compared to [10] and [11] is that, rather than smoothing only the dynamics, it smooths the entire problem by smoothing the value and state-action value functions. This, in turn, implicitly smooths both the cost and the dynamics, allowing it to handle non-smoothness in either.

A. RSOC Formulation

Although the original dynamics in (1) are deterministic, we can formally represent them probabilistically by defining the transition probability

$$p(x_{t+1}|\xi_t) = \delta(x_{t+1} - f_t(\xi_t)), \tag{14a}$$

$$p(x_0) = \delta(x_0 - \mathfrak{x}_0), \tag{14b}$$

[3]Both [10] and [11] use a constant covariance matrix for each horizon timestep. Therefore the subscript t is dropped in the algorithm description

where $\delta(\cdot)$ is the Dirac delta distribution and $\mathfrak{x}_0$ denotes the initial state. The behavior of the controller is characterized by probabilistic policies $\pi_t(u_t|x_t)$, inducing a closed-loop trajectory distribution

$$p(\xi_{0:T};\pi_{0:T-1}) = p(x_0)\prod_{t=0}^{T-1} p(x_{t+1}|\xi_t)\pi_t(u_t|x_t), \quad (15)$$

which evaluates the likelihood of a state-action trajectory $\xi_{0:T}$ under a given policy sequence $\pi_{0:T-1}$.

Introducing an exponential cost transformation, the Risk-Sensitive Stochastic Optimal Control objective can be give as follows:

$$\min_{\pi_{0:T-1}} \underbrace{-\log \mathbb{E}_{p(\xi_{0:T}|\pi_{0:T-1})}\left[\exp\left(-\lambda C\left(\xi_{0:T}\right)\right)\right]}_{\triangleq J(\pi_{0:T-1};\lambda)} \quad (16)$$

where $\lambda > 0$ is the risk-sensitivity parameter.

B. Solution to RSOC via Expectation Maximization

The RSOC problem can be cast as a probabilistic inference problem. In particular, the exponential transform of the cost function (16) allows the control objective to be expressed as a maximum likelihood estimation (MLE) problem on a probabilistic graphical model, where auxiliary optimality variables encode the cost structure [13].

This equivalence enables the use of the Expectation–Maximization (EM) algorithm. EM is a well-established iterative scheme to tackle difficult MLE problems by breaking them down into a sequence of simpler subproblems. Instead of maximizing the likelihood directly, EM alternates between (i) an E-step, where the expected sufficient statistics are computed under the current policy, and (ii) an M-step, where the policy parameters are updated to maximize a surrogate objective. Each iteration improves a lower bound on the likelihood, and the process converges to a (local) optimum. In the following, we present the main results for the iterative scheme, namely the policy update and the risk parameter update that are performed at each EM iteration [18, Theorem 2].

The policy update is given by:

$$\pi_t^*(u_t|x_t) = \pi_t(u_t|x_t)\frac{\exp(-\lambda Q_t^*(\xi_t))}{\exp(-\lambda V_t^*(x_t))} \quad (17)$$

where $\pi(\cdot)$ and $\pi^*(\cdot)$ denote the current and updated policies, respectively, and Q_t^* and V_t^* are the optimal state–action and value functions, following the backward recursion:

$$Q_t^*(\xi_t) = \bar{c}_t(\xi_t) - \tfrac{1}{\lambda}\log \mathbb{E}_{p(x_{t+1}|\xi_t)}\left[\exp(-\lambda V_{t+1}^*(x_{t+1}))\right] \quad (18a)$$

$$V_t^*(x_t) = -\tfrac{1}{\lambda}\log \mathbb{E}_{\pi_t(u_t|x_t)}\left[\exp(-\lambda Q_t^*(\xi_t))\right] \quad (18b)$$

where $\bar{c}_t = c_t - \frac{1}{\lambda}\log\lambda$, and the recursion is initialized with $V_T^* = \bar{c}_T(x_T)$.

The risk sensitivity parameter update is given by:

$$\lambda^* = \frac{T+1}{\mathbb{E}_{p(\xi_{0:T};\pi_{0:T-1}^*)}\left[C(\xi_{0:T})\right]} \quad (19)$$

We remark that optimizing the risk sensitivity parameter is optional. The reformulation of the deterministic TO problem as a MLE problem invites to optimize it, however this is not strictly required nor does not optimizing it impede convergence guarantees.

C. Probabilistic DDP (P-DDP)

Inspired by the DDP algorithm, the policy update in (17) is carried out around a nominal trajectory $(\hat{x}_t, \hat{u}_t)$. The deviation variables $\delta x_t := x_t - \hat{x}_t$ and $\delta u_t := u_t - \hat{u}_t$ are introduced and the policy is parametrized in terms of $(\delta u_t|\delta x_t)$. In this form, the current and updated policies can then be approximated with Gaussians as follows:

$$\pi_t \approx \hat{\pi}_t = \mathcal{N}(\delta u_t;\ k_t + K_t\delta x_t,\ \Sigma_t)\,, \quad (20a)$$

$$\pi_t^* \approx \hat{\pi}_t^* = \mathcal{N}(\delta u_t;\ k_t^* + K_t^*\delta x_t,\ \Sigma_t^*)\,. \quad (20b)$$

The main idea of P-DDP then follows that of standard DDP. A nominal trajectory is first generated by performing a forward closed-loop simulation under the prior probabilistic policy, $\hat{\pi}_{0:T-1}$. This yields a *distribution* over state–action trajectories rather than a single deterministic path. This distribution can then be used in a backward pass to get the smoothed quadratic approximations of the value and state–action value function[4]. The optimal policy is updated according to (17), which can then be used in the forward pass. Finally, the cumulative cost obtained in the forward pass can be used to update the risk sensitivity parameter via (19).

1) Forward pass: To approximate the nominal closed-loop trajectory density $p(\xi_t;\pi_{0:t})$, the uncertainty of an affine prior policy is propagated through the nonlinear dynamics,

$$x_{t+1} = f_t(x_t, \hat{u}_t + \delta u_t), \quad (21a)$$

$$\delta u_t \sim \pi_t(\delta u_t|\delta x_t) \approx \mathcal{N}(\delta u_t; k_t + K_t\delta x_t, \Sigma_{u,t}). \quad (21b)$$

This induces the recursion

$$p(\xi_t;\pi_{0:t}) = p(u_t|x_t;\pi_t)\,p(x_t;\pi_{0:t-1}), \quad (22a)$$

$$p(x_{t+1}|\pi_{0:t}) = \int p(x_{t+1}|\xi_t)\,p(\xi_t;\pi_{0:t})\,d\xi_t, \quad (22b)$$

which is approximated by a Gaussian $p(\xi_t|\pi_{0:t}) \approx \mathcal{N}(\xi_t;\mu_{\xi,t},\Sigma_{\xi\xi,t})$. Starting from $p(x_0) = \mathcal{N}(x_0|\mu_{x,0},\Sigma_{xx,0})$, the updates of the mean $\mu_{\xi,t}$ and covariance $\Sigma_{\xi\xi,t}$ can be given as follows:

$$\mu_{u,t} = \hat{u}_t + k_t + K_t(\mu_{x,t} - \hat{x}_t) \quad (23a)$$

$$\Sigma_{uu,t} = K_t\Sigma_{xx,t}K_t^\top + \Sigma_t \quad (23b)$$

$$\Sigma_{ux,t} = K_t\Sigma_{xx,t} \quad (23c)$$

$$\mu_{x,t+1} = \hat{f}_t(\mu_{\xi,t}) \quad (23d)$$

$$\Sigma_{xx,t+1} = \mathbb{E}\left[(f_t(\mu_{\xi,t} + C_{\xi\xi,t}\epsilon) - \mu_{x,t+1})(\cdot)^\top\right] \quad (23e)$$

[4]Note that the distribution from the forward pass is sufficient for calculating the smoothed estimates in the backward pass; no additional resampling from the dynamics is required, and the computational cost remains same as that of SCS-DDP.

where $\hat{f}_t(\xi_t) := \mathbb{E}[f_t(\xi_t + C_{\xi\xi,t}\epsilon)]$, $\mu_{\xi,t} := \begin{bmatrix}\mu_{x,t}\\ \mu_{u,t}\end{bmatrix}$, $\Sigma_{\xi\xi,t} := \begin{bmatrix}\Sigma_{xx,t} & \Sigma_{xu,t}\\ \Sigma_{ux,t} & \Sigma_{uu,t}\end{bmatrix}$, the random vector $\epsilon \sim \mathcal{N}(0, I)$ is in $\mathbb{R}^{n_x+n_u}$ and $C_{\xi\xi,t}$ is the Cholesky factor of $\Sigma_{\xi\xi,t}$.

2) Backward pass: The smoothed quadratic approximations of the state & state-action value functions are given by

$$\hat{Q}^*_{0,t} := \mathbb{E}[Q^*_t(\mu_{\xi,t} + C_{\xi\xi,t}\epsilon)], \tag{24a}$$

$$\hat{Q}^*_{\xi,t} = \mathbb{E}\Big[Q^*(\mu_{\xi,t} + C_{\xi\xi,t}\epsilon) \otimes C^{-1}_{\xi\xi,t}\epsilon\Big], \tag{24b}$$

$$\hat{Q}^*_{\xi\xi,t} = \mathbb{E}\Big[Q^*_t(\mu_{\xi,t} + C_{\xi\xi,t}\epsilon) \otimes C^{-1}_{\xi\xi,t}(\epsilon\epsilon^\top - I)C^{-1}_{\xi\xi,t}\Big]. \tag{24c}$$

here $Q^*_t(\xi_t) = c_t(\xi_t) + V^*_t(x_t)$, and the backward recursion of the value function is initialized at the terminal time T by:

$$\hat{V}^*_{0,T} := \mathbb{E}[V^*_T(\mu_{x,T} + C_{xx,T}\epsilon)], \tag{25a}$$

$$\hat{V}^*_{x,T} = \mathbb{E}\Big[V^*_T(\mu_{x,T} + C_{xx,T}\epsilon) \otimes C^{-1}_{xx,T}\epsilon\Big], \tag{25b}$$

$$\hat{V}^*_{xx,T} = \mathbb{E}\Big[V^*_T(\mu_{x,T} + C_{xx,T}\epsilon) \otimes C^{-1}_{xx,T}(\epsilon\epsilon^\top - I)C^{-1}_{xx,T}\Big]. \tag{25c}$$

Substituting in (17) and (18), the smoothed quadratic approximations of the value and state-value functions and the Gaussian prior policy (20), yields the following backward-pass updates [18, Lemma 4]:

$$K^*_t = \Sigma^*_t(\Sigma^{-1}_t K_t - \lambda\hat{Q}^*_{ux,t}), \tag{26a}$$

$$k^*_t = \Sigma^*_t(\Sigma^{-1}_t k_t - \lambda\hat{Q}^*_{u,t}), \tag{26b}$$

$$\Sigma^*_t = \left(\Sigma^{-1}_t + \lambda\hat{Q}^*_{uu,t}\right)^{-1}. \tag{26c}$$

and,

$$\hat{V}^*_{x,t} = \hat{Q}^*_{x,t} + \tfrac{1}{\lambda}\Big(K^\top_t\Sigma^{-1}_t k_t - K^{*\top}_t\Sigma^{*,-1}_t k^*_t\Big), \tag{27a}$$

$$\hat{V}^*_{xx,t} = \hat{Q}^*_{xx,t} + \tfrac{1}{\lambda}\Big(K^\top_t\Sigma^{-1}_t K_t - K^{*\top}_t\Sigma^{*,-1}_t K^*_t\Big). \tag{27b}$$

The entire procedure is summarized in Algorithm 2. We would like to point out that this method generates covariance scheduling automatically as a part of the optimization in M-step of the EM algorithm, through the equations (26c) and (19), thus offering a *principled adaptive update*. To avoid covariance collapse of Σ_t, as described in [18], a small regularization term can be added to Σ_t at the end of backward pass in the algorithm iteration. This regularization term can be seen as injecting exploration into the policy, to avoid getting stuck in a local minima.

V. Simulation Experiments

To compare the performance of the discussed three algorithms, we conducted simulation experiments on three simple yet representative non-smooth dynamical systems: **1D Box Push**: where a 1D box constrained to move along a single axis with dry Coulomb friction, is driven from an initial to a goal state; **Pendulum Swing Up**, where a pendulum with joint friction is swung to a target; and **Cartpole Swing Up**, where a cart–pole with joint friction between the pole-cart joint, is driven to a desired configuration.

Algorithm 2: P-DDP algorithm

Input: TO problem: $C(\xi_{0:T}), f_t(\xi_t)$, initial state and covariance: $\mu_{x,0}$, $\Sigma_{xx,0}$, control policy: $\pi_{0:T-1}$, risk-sensitivity parameter: λ, target precision: α^*;
Output: $u^*_{0:T-1}$;
Initialize $\mu_{\xi,0:T}$ and $\Sigma_{\xi\xi,0:T}$ using (23).
repeat
  $k^*_{0:T}, K^*_{0:T}, \Sigma^*_{0:T}$ ▷ Backward Pass using (26);
  $\mu_{\xi,0:T}, \Sigma_{\xi\xi,0:T}$ ▷ Forward Pass using (23);
  $\pi_{0:T-1} \leftarrow \pi^*_{0:T-1}$ ▷ Policy Update;
  $\lambda \leftarrow \lambda^*$ ▷ Optional Risk Sensitivity Parameter Update;
until $|\Delta C| < \alpha^*$;

A. Implementation Details:

Prior to discussing the simulation results, we outline some key implementation aspects of the system dynamics model and the algorithms employed in this study.

All systems are modeled using a time-stepping approach, which first updates velocities through impulse-based increments, then positions through semi-implicit Euler step [30].

Both SCS-DDP and CS-DDP use Monte-Carlo sampling for approximating the expectations in (12) and (13). In our experience, we have found that this can lead to noisy, high variance estimates, which when propagated through the backward pass step, can cause unstable or divergent results. Using antithetic sampling (i.e. every sample's opposite is also part of the sample set) can help reduce this variance, although coming at the cost of doubling the number of dynamics function evaluations. Using sigma points generated via the Gauss-Hermite quadrature rule, on the other hand, provided significantly less noisy estimates of the derivatives, resulting in more reliable backward pass computations without requiring a large increase in function evaluations. Therefore we use sigma points based approximations in all algorithms.

As described in Section (IV-B), updating the risk sensitivity parameter is not strictly necessary, although it can serve as a useful hyperparameter to accelerate the algorithm's convergence. We observed that direct use of (19) is often not effective in practice, as the risk parameter becomes overly sensitive to the absolute cost magnitudes and does not capture a proper trade-off between cost and exploration. Prior work has noted that the scale of cost(/rewards) critically influences the effect of entropy regularization and temperature scheduling (analogous to risk-scheduling in our setting) [31]. Motivated by this, we propose the following updates:

Rule 1: Keep λ constant throughout.

Rule 2: Use the scheme in (19), scaled by a constant $c > 1$. A natural, interpretable choice is to set the scale factor to the cumulative cost after the first forward rollout. With this choice, one gets $\lambda^{k+1} = (T+1)\,\frac{C^0_{0:T}}{C^k_{0:T}}$, where $C^k_{0:T}$ denotes the cumulative cost observed at iteration k, after the forward pass. With this, the update equals $(T+1)$ times the ratio between the initial cost and the current cost – in other words, it directly measures by what factor the cost has been reduced relative to the initial rollout.

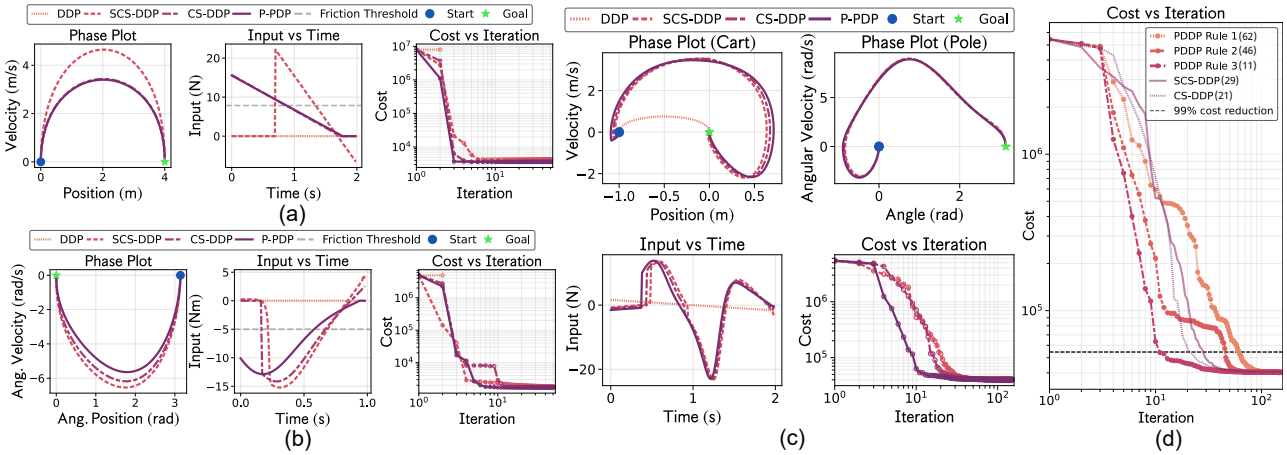

Fig. 3. Comparison of trajectory outcomes and convergence characteristics across multiple trajectory optimization algorithms. (a) Block on ground with friction (b) Pendulum with joint friction (c) Cartpole with joint friction (d) Comparsion of Risk-Sensitivity update rules on Cartpole case (Bracketed values in the legend show iterations required to achieve 99% cost reduction from the initial cost.)

Rule 3: Initialize the risk parameter with the natural choice $\lambda^0 = T + 1$. At each iteration, update λ multiplicatively by a factor $\kappa > 1$ depending on whether the forward rollout reduces the cost, i.e. set $\lambda^{k+1} = \kappa\,\lambda^k$ if $C^k_{0:T} < C^{k-1}_{0:T}$, and $\lambda^{k+1} = \lambda^k/\kappa$ otherwise. Simultaneously, one can also scale the prior precision in the backward pass by a *precision-weighting factor* $\eta > 0$, i.e. replace Σ_t^{-1} by $\eta\,\Sigma_t^{-1}$ in (26) and (27). This factor adjusts the relative weight of the prior covariance versus the curvature information in (26c): $\eta < 1$ down-weights the prior (encouraging exploration), while $\eta > 1$ emphasizes the prior (yielding more conservative updates). In this interpretation, κ can be viewed as the *expected cost reduction factor* per iteration, while η controls how strongly the update adheres to the previous policy.

B. 1D Box Push

We consider the problem of moving a 1 kg block, resting on the ground with a coefficient of friction $\mu = 0.8$, from an initial state $x := (\text{position,velocity}) = (0, 0)$ to a goal state $x_g = (4, 0)$. The TO problem is defined with the following parameters: $W_T = 10^6 I$, $W = 10^{-2} I$, $R = 0.5$, $dt = 0.01$, and $T = 200$. For SCS-DDP, we use the algorithm parameter set: $\Sigma_{xx,0} = 10^{-6} I$, $\Sigma_{uu,0} = 10$, $\alpha_0 = 0.1$, $\gamma = 2$, $\beta = 2$, $\alpha^* = 10^{-6}, \Sigma^*_{xx} = 10^{-6} I$ and $\Sigma^*_{uu} = 10^{-6} I$. For CS-DDP we use the same parameter set as that of SCS-DDP, without $\Sigma_{xx,0}$ and Σ^*_{xx}, and for P-DDP, we use the parameter set: $\Sigma_{xx,0} = 10^{-6} I$, initial control policy $\mathcal{N}(u_t; 0, 10)$, and update the risk sensitivity parameter using Rule 3, with $\lambda^0 = T + 1$, $\kappa = 2$ and $\eta = 0.01$. Sigma points were generated via a 6th-order Gauss–Hermite quadrature rule.

Fig. (3a) show the phase plot, optimized input and the drop in the cost vs algorithm iterations. All algorithms, except the standard DDP, are able to drive the block out of its stiction mode, and reach the goal state, by applying an initial force greater than the friction threshold. SCS-DDP gets trapped in a local minimum owing to exhausted exploration.

C. Pendulum Swing Up

The experiment aims to swing up a single pendulum, with length 1 m and a mass 1 kg attached to its end, from an initial downward resting position $x = (\pi, 0)$ to the swung up state $x_g = (0, 0)$. A static friction threshold of 5 Nm is considered in the pendulum joint. The TO problem uses: $W_T = 10^6 I$, $W = 10^{-2} I$, $R = 0.5$, $dt = 0.01$, and $T = 100$. For all three algorithms, we use the same parameter set as used in Sec. (V-B), except $\Sigma_{uu,0} = 20$ for SCS-DDP and CS-DDP, and an initial control policy $\mathcal{N}(u_t; 0, 20)$ for P-DDP.

As can be seen in Fig. (3b), again, all algorithms, except the standard DDP, are able to drive the pendulum out of its stiction mode, and reach the goal state, by applying an initial force greater than the friction threshold. Also, both SCS-DDP and CS-DDP, can be seen getting trapped in a local minimum due to fully depleted exploration.

D. Cartpole Swing Up

The experiment aims to drive an underactuated cartpole from the resting state $x = (-1, 0, 0, 0)$ to the swung-up state $x_g = (0, 0, \pi, 0)$, where the first two coordinates denote the cart's position and velocity, and the last two denote the pole's angular position and velocity. The control input is a force on the cart. System parameters are: cart mass 1 kg, pole mass 0.1 kg, pole length 0.5 m, and static friction threshold 0.2 Nm at the cart–pole joint. The trajectory optimization problem uses $W_T = 10^6 I$, $W = 10^{-2} I$, $R = 5$, $dt = 0.01$, and $T = 200$. All algorithms use the same parameters as in Sec. (V-B), except $\Sigma_{uu,0} = 15$ for SCS-DDP and CS-DDP, and an initial control policy $\mathcal{N}(u_t; 0, 15)$ for P-DDP. We used a 5th-order Gauss–Hermite quadrature rule.

Fig. (3c) shows how the DDP algorithm is stuck at a local minima, wherein only the cart gets driven to the goal state, while the other three algorithms are able to also drive the pole out of its stiction zone and to its goal state. Fig. (3d) shows the convergence characteristics of the additional two update rules described in Section V-A. As can be seen, Rule 3 outperforms others with its explore-exploit feature.

VI. Discussion

In this work we discussed three algorithms dedicated to TO of non-smooth dynamical systems. Across our benchmarks, all were successful in navigating out of stiction

mode and converged to acceptable behavior, whereas standard DDP failed in each setting. To accomplish this, the methods introduce stochastic smoothing, but each does so in its own distinct way. The key differences lie in *where* and *how* this smoothing is applied. SCS-DDP performs smoothing in both state and control spaces, which can provide more stable derivative estimates at the cost of higher sample complexity. CS-DDP restricts smoothing to the input dimension, thereby reducing computational demands and ensuring that only reachable states are explored. However, this efficiency comes with the practical requirement that derivatives of the dynamics function with respect to the state must be available. This may not always be the case. Finally, P-DDP roots the smoothing mechanism directly within the probabilistic optimal control framework, through a probabilistic reformulation of a deterministic TO problem. This effectively smooths the entire problem at the level of the (state-action) value functions, thus also allowing non-smooth cost functions. Moreover, P-DDP provides *principled* scheduling of the noise as part of the EM update scheme. In this work, we further extended P-DDP by successfully applying it to non-smooth robotic tasks and augmenting its stochasticity scheduling with practical update schemes.

VII. Future Work

While we demonstrated the algorithms on simple illustrative systems, our future work will scale them to high-dimensional, contact-rich tasks such as dexterous manipulation and legged locomotion. A key open problem is developing a principled and automatic update scheme for the risk sensitivity parameter, since current heuristics (e.g., scaling rules, multiplicative updates) only partially balance exploration and exploitation. Incorporating hard state and input constraints is another important direction. Moreover, as the P-DDP method is rooted in a probabilistic optimal control framework, it naturally enables extensions to uncertainty quantification, maximum-entropy methods, and uncertainty-aware trajectory optimization. Finally, a systematic theoretical comparison of smoothing-based approaches—considering computational cost, variance, convergence properties and guarantees—would clarify their relative strengths and guide practical use in robotics.

References

[1] D. H. Jacobson and D. Q. Mayne, *Differential Dynamic Programming*. New York: American Elsevier Pub. Co., 1970.

[2] J. Z. Zhang, T. A. Howell, Z. Yi, C. Pan, G. Shi, G. Qu, T. Erez, Y. Tassa, and Z. Manchester, "Whole-body model-predictive control of legged robots with mujoco," *arXiv preprint arXiv:2503.04613*, 2025.

[3] Y. Tassa, N. Mansard, and E. Todorov, "Control-limited differential dynamic programming," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2014, pp. 1168–1175.

[4] T. A. Howell, B. E. Jackson, and Z. Manchester, "Altro: A fast solver for constrained trajectory optimization," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2019, pp. 7674–7679.

[5] K. Werling, D. Omens, J. Lee, I. Exarchos, and C. K. Liu, "Fast and feature-complete differentiable physics for articulated rigid bodies with contact," in *Proc. Robotics: Sci. Syst. (RSS)*, 2021.

[6] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, and S. Levine, "Learning complex dexterous manipulation with deep reinforcement learning and demonstrations," *arXiv preprint arXiv:1709.10087*, 2017.

[7] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile and dynamic motor skills for legged robots," *Sci. Robot.*, vol. 4, p. eaau5872, 2019.

[8] H. J. T. Suh, T. Pang, and R. Tedrake, "Bundled gradients through contact via randomized smoothing," *IEEE Robot. Autom. Lett.*, vol. 7, pp. 4000–4007, 2022.

[9] C.-E. Sun, S. Gao, and T.-W. Weng, "Breaking the barrier: Enhanced utility and robustness in smoothed drl agents," in *Proc. 41st Int. Conf. Mach. Learn. (ICML), PMLR*, vol. 235, 2024, pp. 46 957–46 987.

[10] T. Pang, H. J. T. Suh, L. Yang, and R. Tedrake, "Global planning for contact-rich manipulation via local smoothing of quasi-dynamic contact models," *IEEE Trans. Robot.*, vol. 39, pp. 4691–4711, 2023.

[11] Q. L. Lidec, F. Schramm, L. Montaut, C. Schmid, I. Laptev, and J. Carpentier, "Leveraging randomized smoothing for optimal control of nonsmooth dynamical systems," *Nonlinear Anal. Hybrid Syst.*, vol. 52, p. 101468, 2024.

[12] S. S. Hassan and S. Särkkä, "Fourier-hermite dynamic programming for optimal control," *IEEE Trans. Autom. Control*, vol. 68, pp. 6377–6384, 2023.

[13] S. Levine, "Reinforcement learning and control as probabilistic inference: Tutorial and review," *arXiv preprint arXiv:1805.00909*, 2018.

[14] T. Lefebvre, "Probabilistic control and majorisation of optimal control," *Syst. Control Lett.*, vol. 190, p. 105837, 2024.

[15] K. C. Rawlik, "On probabilistic inference approaches to stochastic optimal control," Ph.D. dissertation, University of Edinburgh, 2013.

[16] J. Watson and J. Peters, "Advancing trajectory optimization with approximate inference: Exploration, covariance control and adaptive risk," in *2021 American Control Conference (ACC)*. IEEE, 2021, pp. 1231–1236.

[17] S. Levine and V. Koltun, "Variational policy search via trajectory optimization," *Advances in neural information processing systems*, vol. 26, 2013.

[18] M. M. Filabadi, T. Lefebvre, and G. Crevecoeur, "Deterministic trajectory optimization through probabilistic optimal control," *arXiv preprint arXiv:2407.13316*, 2024.

[19] Y. Aoyama, O. So, A. D. Saravanos, and E. A. Theodorou, "Second-order constrained dynamic optimization," *arXiv preprint arXiv:2409.11649*, 2024.

[20] T. A. Howell, S. L. Cleac'h, J. Z. Kolter, M. Schwager, and Z. Manchester, "Dojo: A differentiable simulator for robotics," *arXiv preprint arXiv:2203.00806*, vol. 9, p. 4, 2022.

[21] J. V. Burke, A. S. Lewis, and M. L. Overton, "Approximating subdifferentials by random sampling of gradients," *Math. Oper. Res.*, vol. 27, pp. 567–584, 2002.

[22] J. Burke, F. Curtis, A. Lewis, M. Overton, and L. Simões, "Gradient sampling methods for nonsmooth optimization," *Numerical Nonsmooth Optim.: State Art Algorithms*, pp. 201–225, 2020.

[23] J. Duchi, M. Jordan, M. Wainwright, and A. Wibisono, "Optimal rates for zero-order convex optimization: The power of two function evaluations," *IEEE Trans. Inf. Theory*, vol. 61, pp. 2788–2806, 2015.

[24] J. Zhu, "Hessian estimation via stein's identity in black-box problems," in *Proc. 2nd Math. Sci. Mach. Learn. Conf. (PMLR)*, vol. 145, 2022, pp. 1161–1178.

[25] Q. L. Lidec, W. Jallet, L. Montaut, I. Laptev, C. Schmid, and J. Carpentier, "Contact models in robotics: A comparative analysis," *IEEE Trans. Robot.*, vol. 40, pp. 3716–3733, 2024.

[26] P. Varin and S. Kuindersma, "A constrained kalman filter for rigid body systems with frictional contact," in *Int. Workshop Algor. Found. Robot. (WAFR)*, 2018, pp. 474–490.

[27] J. Carpentier, Q. L. Lidec, and L. Montaut, "From compliant to rigid contact simulation: A unified and efficient approach," in *Proc. Robotics: Sci. Syst. (RSS)*, 2024.

[28] Q. L. Lidec, L. Montaut, Y. de Mont-Marin, F. Schramm, and J. Carpentier, "End-to-end and highly-efficient differentiable simulation for robotics," *arXiv preprint arXiv:2409.07107*, 2024.

[29] H. Robbins and S. Monro, "A stochastic approximation method," *Ann. Math. Statist.*, pp. 400–407, 1951.

[30] D. Stewart and J. C. Trinkle, "An implicit time-stepping scheme for rigid body dynamics with coulomb friction," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2000, pp. 162–169.

[31] B. Belousov and J. Peters, "Entropic regularization of markov decision processes," *Entropy*, vol. 21, p. 674, 2019.